

1.INTRODUCTION TO FILE ALLOCATION

A file system is one of the most visible services provided by an operating system. Users work with names such as document.txt or photo.jpg, while the operating system manages the actual storage blocks underneath those names. When a file is created or extended, the file system has to identify suitable free blocks and record how those blocks belong to the file.

File allocation means deciding the physical placement of a file's blocks on a storage device. The allocation method affects access performance, space utilization, file growth, and the amount metadata required. Standard operating-system material identifies contiguous, linked, and indexed allocation as the three classical approaches.

The basic unit of allocation is usually a fixed-size block. If a file needs 20 blocks, the file system must associate 20 physical blocks with that file. The blocks may be adjacent, scattered and linked, or scattered with their addresses collected in an index structure.

A good allocation strategy should make efficient use of storage, support the required access pattern, minimize unnecessary overhead, and allow files to grow without excessive relocation.

2. OBJECTIVES AND SCOPE

The main objective of this micro project is to study how an operating system allocates storage blocks to files and to understand the strengths and weaknesses of the classical allocation methods.

- Explain the purpose of file allocation in an operating system.
- Understand how logical files are mapped to physical disk blocks.
- Study contiguous allocation and its impact on access speed and fragmentation.
- Study linked allocation and the role of pointers between blocks.
- Study indexed allocation and the use of an index block for direct access.
- Compare methods using access speed, space utilization, file growth and complexity.
- Understand the relationship between allocation and free-space management.
- Develop a simple algorithmic model suitable for classroom implementation.
- Identify practical situations where different allocation strategies are useful.

Scope: The project focuses on classical allocation concepts taught in Operating System courses. Modern systems may use extents, multi-level indexing, journaling, caching and other mechanisms, but these can be understood more easily after mastering the basic models.

3. FILE SYSTEM ARCHITECTURE

A simplified file system can be viewed as a set of layers. Applications request operations such as create, open, read, write and delete. The operating system translates these logical operations into file-system actions. Metadata records the file name, size, permissions, timestamps and the information required to locate its blocks.

The allocation manager determines which physical blocks are assigned to the file. The free-space manager records blocks available for future allocation. Directory structures connect human-readable file names to file metadata.

The physical storage layer contains actual data and metadata blocks. On traditional disks, placement can influence seek behavior. On SSDs, there is no mechanical head movement, but allocation and metadata still affect storage efficiency and write behavior.

Component	Main Responsibility
Application	Requests file operations
File-system interface	Provides create/open/read/write/delete services
Directory & metadata	Stores file identity and allocation information
Allocation manager	Maps file blocks to storage blocks
Free-space manager	Tracks unused blocks
Storage device	Stores data and metadata blocks

4. CONTIGUOUS ALLOCATION

In contiguous allocation, all blocks of a file are placed next to one another in a continuous range. A directory entry can identify the starting block and the length of the allocated region. For a file beginning at block b and occupying n blocks, the physical blocks are $b, b+1, \dots, b+n-1$. This supports sequential and direct access efficiently.

Example: If File A requires five blocks and begins at block 4, it occupies blocks 4, 5, 6, 7 and 8. The physical address of a logical block can be calculated from the starting address plus its logical offset.

- Fast sequential access because neighboring file blocks are stored together.
- Fast direct access because a logical block can be converted into a physical block.
- Low metadata overhead because start and length can describe the allocation.
- Simple concept and implementation.

The main difficulty is file growth. If the blocks immediately after the file are occupied, the file may need to be moved to a larger contiguous region. Repeated creation and deletion can also produce external fragmentation.

5. LINKED ALLOCATION

Linked allocation stores a file as a linked list of disk blocks. The blocks do not have to be physically adjacent. Each block contains data and a pointer to the next block, while directory information keeps track of the beginning of the chain and, in many designs, the ending block.

For example, a file may occupy blocks $7 \rightarrow 12 \rightarrow 3 \rightarrow 10 \rightarrow 15$. The physical positions are scattered, but the pointers define the logical order.

- External fragmentation is avoided because blocks can be allocated anywhere.
- File size can grow dynamically as additional free blocks become available.
- The allocator does not need one large contiguous region.
- Sequential access is natural because blocks are followed in pointer order.

The main weakness is random access. To reach a later block, the system may have to follow the chain from the first block. Pointer fields also consume storage space, and pointer corruption can break the chain.

6. INDEXED ALLOCATION

Indexed allocation separates the list of block addresses from the file data. Each file has an index block containing addresses of its data blocks. The directory entry points to the index block, and the i -th entry identifies the i -th data block.

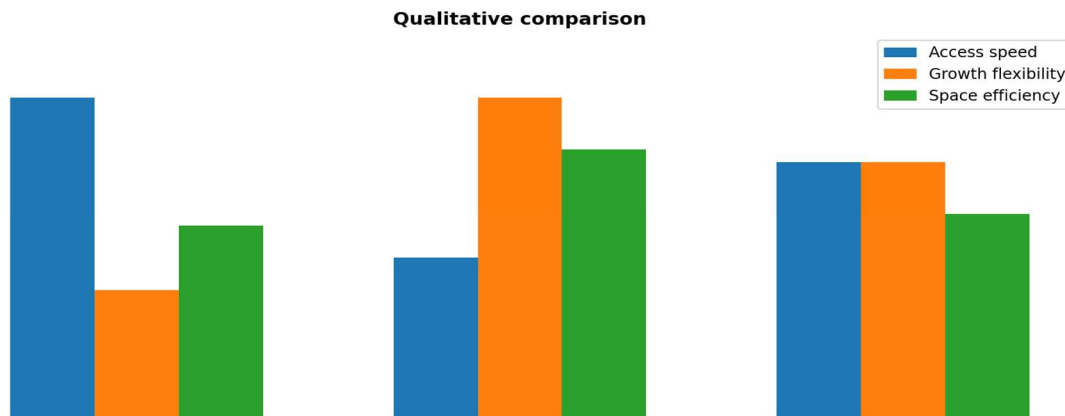
Suppose the index block contains [7, 12, 3, 10, 15]. The first logical block is stored at physical block 7, the second at 12, and so on. The data blocks can be scattered, so external fragmentation is avoided. Direct access is supported because the required block address can be obtained from the index.

- Supports sequential and direct access.
- Does not require file blocks to be contiguous.
- Files can grow by adding block addresses to the index structure.
- Provides a clear mapping between logical block numbers and physical blocks.

The principal disadvantage is index overhead. A small file may still require an index block. Very large files can require linked or multi-level index structures, increasing complexity.

7. ALLOCATION METHODS

The three classical allocation methods make different trade-offs. Contiguous allocation emphasizes speed and simple addressing. Linked allocation emphasizes flexible growth and avoidance of external fragmentation. Indexed allocation aims to combine non-contiguous placement with efficient direct access.



Feature	Contiguous	Linked	Indexed
Block placement	Adjacent	Scattered	Scattered
Sequential access	Excellent	Good	Good
Direct/random access	Excellent	Poor	Good
File growth	Difficult	Easy	Easy
External fragmentation	Possible	No	No
Pointer/index overhead	Very low	Per-block pointer	Index block(s)
Implementation complexity	Low	Medium	Medium–High
Best suited for	Fast, stable files	Sequential/growing files	General direct-access needs

The comparison is qualitative rather than a benchmark. Actual performance depends on the storage device, caching, workload, file size, allocation policy and implementation details

8. FREE-SPACE MANAGEMENT

File allocation cannot work without knowing which storage blocks are free. A free-space manager records available blocks and updates this information whenever blocks are allocated or released. Common approaches include bit vectors, linked free lists, grouping and counting.

Technique	Basic Idea	Strength	Limitation
Bit vector	One bit represents each block	Compact and easy to inspect	Needs one bit per block
Linked list	Free blocks are linked	Simple concept	Searching may be slow
Grouping	One free block stores many free-block addresses	Reduces traversal	More complex
Counting	Stores start address and count of consecutive free blocks	Compact for runs	Less useful when space is scattered

A bit vector is especially intuitive: a bit can represent whether a block is free or allocated. Free-space management is closely connected with fragmentation. When files are created and deleted, free blocks can become scattered, which is particularly important for contiguous allocation.

Standard operating-system material commonly introduces bit vectors, linked lists, grouping and counting as basic free-space strategies

9. ALGORITHM

A simplified allocation procedure first determines the number of blocks required, checks free-space information, selects an allocation method, assigns suitable blocks and updates file metadata.

ALLOCATE_FILE(file, required_blocks):

- determine number of blocks needed

- check free-space structure

- if insufficient free blocks:

 - report allocation failure

- else:

 - choose allocation strategy

 - if CONTIGUOUS:

 - find one free run of required_blocks

 - reserve the run

 - if LINKED:

 - choose required free blocks

 - link each block to the next

 - if INDEXED:

 - reserve an index block

 - choose required free blocks

 - store their addresses in the index

- update file metadata

- return allocation information

For a classroom implementation, the disk can be represented by an array `block[0...N-1]`, where each entry records whether a block is free or allocated. A file record can store its name, size, method, starting block or index block, and the list of allocated blocks.

This simplified model demonstrates allocation decisions without implementing a complete kernel-level file system.

10. ADVANTAGES

A File Allocation System helps the operating system use available disk space efficiently. It decides where the blocks of a file should be stored and keeps track of occupied and free blocks. This reduces unnecessary wastage of storage space.

One of the major advantages is faster access to stored files. Depending on the allocation method used, the operating system can quickly locate the blocks containing a particular file. Contiguous allocation, for example, provides very fast access because blocks are stored next to each other.

The system maintains files in an organized manner on secondary storage. It records information about the location of file blocks, making it easier for the operating system to locate and manage files.

Efficient allocation reduces unnecessary disk operations. When files are stored and accessed efficiently, the overall performance of the operating system can improve.

File allocation keeps track of used and free blocks, helping the operating system use available storage space effectively.

Proper allocation and tracking of file blocks reduce the chances of losing file information due to incorrect block management.

11.DISADVANTAGES

Although a file allocation system provides efficient file management, it also has some limitations. The disadvantages depend on the type of allocation method used.

In contiguous allocation, free space may become divided into small pieces. This can make it difficult to allocate space for large files.

Linked and indexed allocation require extra storage for pointers or index blocks. This increases the overhead of the file system.

Indexed allocation requires index blocks to store addresses of file blocks, which consumes additional disk space.

A good allocation method allows files to be accessed quickly. For example, contiguous allocation provides very fast sequential and direct access.

File allocation methods help the operating system manage available disk space efficiently. Files are divided into blocks and stored according to the selected allocation method.

A File Allocation System is a method used by an operating system to organize and store files on secondary storage devices such as hard disks, SSDs, and USB drives. It keeps track of where the blocks of each file are located and helps the system access files efficiently.

Different allocation techniques such as contiguous, linked, and indexed allocation can be used depending on system requirements

12. APPLICATIONS

File allocation concepts appear in many storage workloads. The classical methods are educational models, while real file systems often combine or extend them.

Scenario	Useful Allocation Idea	Reason
Large sequential data	Contiguous / extent-like placement	Efficient sequential transfer
Growing log file	Linked-style or extent-based growth	Blocks can be added as needed
Database-style random access	Indexed or multi-level indexing	Fast access to selected blocks
Read-mostly fixed files	Contiguous placement	Stable placement and efficient reads
Small changing files	Flexible indexed/metadata structures	Avoid repeated large relocations

Example: Consider a 100-block file. Under contiguous allocation, the file could occupy blocks 500–599. Under linked allocation, its 100 blocks could be distributed across free locations, with pointers maintaining order. Under indexed allocation, an index structure would store the 100 physical addresses.

Modern systems may use extent-based allocation, where a file is described using ranges of contiguous blocks rather than a single range. This preserves many benefits of contiguous placement while allowing a file to grow through multiple extents.

13. CASE STUDY

Assume a disk contains 40 blocks numbered 0 to 39. Three files are created: File A requires 5 blocks, File B requires 4 blocks, and File C requires 6 blocks.

Contiguous case: if blocks 0–4 are free, File A can be placed there. File B can use blocks 5–8 and File C can use blocks 9–14. If File B is deleted, blocks 5–8 become a hole. A new file requiring 7 contiguous blocks cannot use that hole even though four blocks are free there.

Linked case: File A might use 0 → 12 → 5 → 21 → 8. File B could use 4 → 19 → 7 → 30. File C could use 2 → 16 → 28 → 11 → 33 → 6. The blocks are scattered, but pointers preserve file order.

Indexed case: File A receives an index block containing [0,12,5,21,8]. File B receives another index block containing [4,19,7,30]. The file system can locate a logical block through its index entry without traversing all earlier blocks.

Question	Contiguous	Linked	Indexed
Can blocks be scattered?	No	Yes	Yes
Easy direct access?	Yes	No	Yes
Easy file growth?	No	Yes	Yes
External fragmentation?	Possible	No	No

14. PERFORMANCE

File allocation performance should not be judged only by raw access speed. A practical design considers allocation overhead, metadata size, file growth, fragmentation, reliability, caching, workload patterns and storage-device characteristics.

- Access pattern: sequential workloads benefit from ordered blocks.
- Random access: direct block mapping reduces traversal of earlier blocks.
- File growth: frequently growing files benefit from flexible allocation.
- Fragmentation: the file system should avoid unusable free-space fragments.
- Metadata overhead: pointers and index blocks consume storage and may require additional reads.
- Reliability: allocation metadata must remain consistent after failures and recovery.
- Scalability: large files may require multi-level indexing or extent structures.
- Storage technology: HDDs and SSDs have different physical characteristics.

The classic allocation methods illustrate these trade-offs clearly. Contiguous allocation minimizes mapping complexity, linked allocation minimizes the need for contiguous free space, and indexed allocation provides a flexible address map for direct access.

Modern file systems frequently use advanced hybrids rather than relying on one classical method exactly as described in textbooks. The classical models remain valuable because they explain the fundamental design choices behind those systems.

15. FUTURE SCOPE

This micro project uses simplified diagrams and conceptual algorithms. A production file system contains additional mechanisms, including permissions, directory indexing, caching, consistency rules, crash recovery, journaling or copy-on-write techniques, quotas, security controls and device-specific optimizations.

The project treats storage as a simple logical block array. Real storage devices expose more complicated behavior. SSDs use flash translation layers and have erase/write characteristics different from magnetic disks. Therefore, the allocation method visible at the file-system level does not always correspond directly to physical flash-cell placement.

Future scope:

- Implement a graphical simulator for all three allocation methods.
- Show real-time allocation and deallocation of blocks.
- Add a fragmentation calculator.
- Compare simulated access counts for sequential and random workloads.
- Implement multi-level indexed allocation.
- Add extent-based allocation as an advanced method.
- Create a visual file-system block map.
- Add crash simulation to demonstrate metadata consistency and recovery.

Such extensions could turn the micro project into an educational file-system simulator suitable for laboratory demonstration.

16. CONCLUSION

File allocation is a fundamental part of operating-system file-system management. It determines how logical file blocks are mapped onto physical storage blocks and influences access speed, storage utilization, file growth and metadata overhead.

Contiguous allocation is simple and fast because a file occupies consecutive blocks. Its main weaknesses are fragmentation and difficult file growth. Linked allocation removes the need for contiguous space and allows easy growth, but introduces pointer overhead and makes random access inefficient. Indexed allocation stores block addresses in an index structure, providing direct access while allowing blocks to be scattered, but requires additional index storage and can become more complex for very large files.

The important lesson is that file allocation is a trade-off problem. A good file-system design selects or combines allocation techniques according to workload, storage technology, file size, growth behavior and access pattern. Understanding the three classical methods provides a strong foundation for studying modern file systems and storage management.

Overall, the File Allocation System demonstrates how operating systems transform a simple user-level concept—a file—into an organized collection of storage blocks managed through metadata and allocation structures.

17. REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts, 9th Edition, John Wiley & Sons, 2012.
 2. Tanenbaum, A. S., & Bos, H. Modern Operating Systems, 4th Edition, Pearson, 2014.
 3. Nutt, G. Operating Systems, 3rd Edition, Pearson, 2009.
 4. Mary Anitha Rajam, “File Allocation Methods – Operating System,” INFLIBNET e-PG Pathshala.
 5. MIT, Operating Systems Lecture Notes, Lecture 13: File System Implementation
 6. GeeksforGeeks, “File Allocation Methods,” updated April 15, 2026.
 7. Loyola University Chicago, Introduction to Operating Systems: Principles and Practice, “Implementing Files and Folders,” 2026.
 8. CS Engineering Gyan, “File Allocation Methods in Operating System,” educational reference
- Reference note: The diagrams in this project are original simplified illustrations prepared for academic presentation. The references are used for conceptual grounding.